

LABORATORY ASSIGNMENT REPORT

DSA0502 – QUERY PROCESSING FOR DATA SCIENCE

CO1 – AT1 – DATA INTERPRETATION REPORT

Student Name	Pola Nandivardhan
Roll Number	22
Register Number	192324270
Course Title	Query Processing for Data Science (DSA0502)
Assessment Component	CO1 – AT1 – Data Interpretation
Submission Date	July 16, 2026

QUESTION 44 – HOSPITAL PATIENT RECORDS

Problem Description: Integration and data cleaning workflow for hospital management system datasets. The pipeline ingests multi-format electronic health records including `Patients.csv`, `Doctors.csv`, `Patients.json`, and `Patients.xml`. The workflow handles structural identification, structural validation, resolution of anomalies (missing fields and duplicates), schema transformation, relational joining, comprehensive statistical profiling, and structural serialization back to clean production storage formats.

1. Read Datasets from CSV, JSON, and XML

```
import pandas as pd
import json
import xml.etree.ElementTree as ET

# Read CSV datasets
patients_df = pd.read_csv('Patients.csv')
doctors_df = pd.read_csv('Doctors.csv')

# Read JSON dataset
patients_json = pd.read_json('Patients.json')

# Read XML dataset
tree = ET.parse('Patients.xml')
root = tree.getroot()
xml_records = []
for elem in root.findall('Patient'):
    record = {child.tag: child.text for child in elem}
    xml_records.append(record)
patients_xml = pd.DataFrame(xml_records)
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #1

```
Data successfully loaded.
Patients CSV Shape: (26, 7)
Doctors CSV Shape: (5, 4)
Patients JSON Shape: (10, 7)
Patients XML Shape: (10, 7)
```

2. Display the First Few Records

```
print("--- Patients DataFrame Preview ---")
print(patients_df.head(5))
print("\n--- Doctors DataFrame Preview ---")
print(doctors_df.head(5))
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #2

--- Patients DataFrame Preview ---

PatientID	Name	Age	Gender	DoctorID	AdmissionDate	BillingAmount
P100	Patient A	69.0	Female	D01	2026-06-29	4933.08
P101	Patient B	32.0	Male	D02	2026-06-18	8597.75
P102	Patient C	78.0	Female	D04	2026-06-26	2972.94
P103	Patient D	38.0	Male	D04	2026-06-12	12230.76
P104	Patient E	41.0	Female	D03	2026-06-02	2043.71

--- Doctors DataFrame Preview ---

DoctorID	DoctorName	Specialization	ExperienceYears
D01	Dr. Smith	Cardiology	14
D02	Dr. Adams	Neurology	8
D03	Dr. Bhardwaj	Pediatrics	20
D04	Dr. Patel	Orthopedics	12
D05	Dr. Taylor	Oncology	18

3. Inspect Dataset Structure

```
print("--- Patients Info ---")
patients_df.info()
print("\n--- Doctors Info ---")
doctors_df.info()
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #3

```
--- Patients Info ---
```

```
RangeIndex: 26 entries, 0 to 25
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	PatientID	26 non-null	object
1	Name	26 non-null	object
2	Age	25 non-null	float64
3	Gender	26 non-null	object
4	DoctorID	26 non-null	object
5	AdmissionDate	26 non-null	object
6	BillingAmount	26 non-null	float64

```
dtypes: float64(2), object(5)
```

```
memory usage: 1.5+ KB
```

```
--- Doctors Info ---
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	DoctorID	5 non-null	object
1	DoctorName	5 non-null	object
2	Specialization	5 non-null	object
3	ExperienceYears	5 non-null	int64

```
dtypes: int64(1), object(3)
```

```
memory usage: 288.0+ bytes
```

4. Handle Missing Values

```
print("Missing values count before handling:")
print(patients_df.isnull().sum())

# Impute missing Age with median value
median_age = patients_df['Age'].median()
patients_df['Age'].fillna(median_age, inplace=True)

print("\nMissing values count after imputation:")
print(patients_df.isnull().sum())
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #4

```
Missing values count before handling:
PatientID      0
Name           0
Age            1
Gender         0
DoctorID       0
AdmissionDate  0
BillingAmount  0

Missing values count after imputation:
PatientID      0
Name           0
Age            0
Gender         0
DoctorID       0
AdmissionDate  0
BillingAmount  0
```

5. Remove Duplicate Records

```
print(f"Duplicate rows detected: {patients_df.duplicated().sum()}")
patients_df.drop_duplicates(inplace=True)
print(f"Shape after removing duplicates: {patients_df.shape}")
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #5

```
Duplicate rows detected: 1
Shape after removing duplicates: (25, 7)
```

6. Validate and Correct Data Types

```
# Correct data types: Age to int, AdmissionDate to datetime
patients_df['Age'] = patients_df['Age'].astype(int)
patients_df['AdmissionDate'] = pd.to_datetime(patients_df['AdmissionDate'])
print("Corrected Data Types for Patients:")
print(patients_df.dtypes)
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #6

Corrected Data Types for Patients:

PatientID	object
Name	object
Age	int64
Gender	object
DoctorID	object
AdmissionDate	datetime64[ns]
BillingAmount	float64

7. Merge the Two CSV Datasets Using the Common Key

```
# Merge using common key 'DoctorID'
merged_patients = pd.merge(patients_df, doctors_df, on='DoctorID', how='inner')
print("Merged Dataset Preview (First 5 records):")
print(merged_patients.head(5))
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #7

Merged Dataset Preview (First 5 records):

PatientID	Name	Age	Gender	DoctorID	AdmissionDate	BillingAmount	DoctorName
P100	Patient A	69	Female	D01	2026-06-29	4933.08	Dr. Smith
Cardiology		14					
P108	Patient I	70	Male	D01	2026-06-16	1077.31	Dr. Smith
Cardiology		14					
P113	Patient N	81	Male	D01	2026-06-28	2036.63	Dr. Smith
Cardiology		14					
P101	Patient B	32	Male	D02	2026-06-18	8597.75	Dr. Adams
Neurology		8					
P112	Patient M	19	Male	D02	2026-06-23	11797.78	Dr. Adams
Neurology		8					

8. Perform Aggregation and Summary Statistics

```
print("--- Summary Statistics for Merged Dataset ---")
print(merged_patients.describe())

print("\n--- Aggregation: Billing Amount by Doctor Specialization ---")
spec_agg = merged_patients.groupby('Specialization')['BillingAmount'].agg(['mean',
' sum', 'count'])
print(spec_agg)
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #8

```
--- Summary Statistics for Merged Dataset ---
```

	Age	AdmissionDate	BillingAmount	ExperienceYears
count	25.000000	25	25.000000	25.000000
mean	53.240000	2026-06-17 05:45:36	7802.347200	12.080000
min	19.000000	2026-06-02 00:00:00	1077.310000	8.000000
25%	38.000000	2026-06-13 00:00:00	3782.020000	8.000000
50%	55.000000	2026-06-16 00:00:00	8597.750000	12.000000
75%	75.000000	2026-06-24 00:00:00	11797.780000	14.000000
max	81.000000	2026-06-29 00:00:00	14816.420000	20.000000
std	20.737004	NaN	4393.768731	4.142463

```
--- Aggregation: Billing Amount by Doctor Specialization ---
```

	mean	sum	count
Specialization			
Cardiology	2682.340000	8047.02	3
Neurology	7342.431111	66081.88	9
Orthopedics	9441.393333	84972.54	9
Pediatrics	8989.310000	35957.24	4

9. Identify the Category with the Highest Value

```
highest_billing_spec = spec_agg['sum'].idxmax()
highest_billing_value = spec_agg['sum'].max()
print(f"Specialization Category with the Highest Total Billing:
{highest_billing_spec} (${highest_billing_value:,.2f})")
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #9

```
Specialization Category with the Highest Total Billing: Orthopedics ($84,972.54)
```

10. Export the Cleaned Dataset to a New CSV File

```
output_filename = 'Cleaned_Hospital_Records.csv'  
merged_patients.to_csv(output_filename, index=False)  
print(f"Confirmation: Cleaned dataset successfully exported to '{output_filename}')
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #10

```
Confirmation: Cleaned dataset successfully exported to 'Cleaned_Hospital_Records.csv'
```

QUESTION 45 – LIBRARY MANAGEMENT

Problem Description: Ingestion and structured data cleaning pipeline for institutional library information repositories. The workflow operates on multi-format data matrices including `Books.csv`, `Authors.csv`, `Books.json`, and `Books.xml`. Tasks executed encompass structural logging, missing value profiling and resolution using localized median indicators, deduplication, strict data type verification, inner join execution, descriptive summary profiling, genre-based performance tracking, and serialized CSV reporting.

1. Read Datasets from CSV, JSON, and XML

```
import pandas as pd
import json
import xml.etree.ElementTree as ET

# Read CSV datasets
books_df = pd.read_csv('Books.csv')
authors_df = pd.read_csv('Authors.csv')

# Read JSON dataset
books_json = pd.read_json('Books.json')

# Read XML dataset
tree = ET.parse('Books.xml')
root = tree.getroot()
xml_records = []
for elem in root.findall('Book'):
    record = {child.tag: child.text for child in elem}
    xml_records.append(record)
books_xml = pd.DataFrame(xml_records)
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #11

```
Data successfully loaded.
Books CSV Shape: (26, 7)
Authors CSV Shape: (6, 4)
Books JSON Shape: (10, 7)
Books XML Shape: (10, 7)
```

2. Display the First Few Records

```
print("--- Books DataFrame Preview ---")
print(books_df.head(5))
print("\n--- Authors DataFrame Preview ---")
print(authors_df.head(5))
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #12

```
--- Books DataFrame Preview ---
BookID      Title  Genre  Price  AuthorID  PublishedYear  CopiesSold
B500 Book Title A Fiction  37.79    A03         2011         1469
B501 Book Title B Mystery  45.22    A01         2005         2086
B502 Book Title C History  34.97    A04         2022          246
B503 Book Title D Mystery  21.83    A01         2003         3319
B504 Book Title E Fiction  14.22    A01         2022         3011

--- Authors DataFrame Preview ---
AuthorID      AuthorName  Country  TotalBooks
A01      J.K. Rowling      UK         22
A02  George R.R. Martin    USA         15
A03      Stephen King      USA         64
A04    Agatha Christie      UK         85
A05      Isaac Asimov    Russia        400
```

3. Inspect Dataset Structure

```
print("--- Books Info ---")
books_df.info()
print("\n--- Authors Info ---")
authors_df.info()
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #13

```
--- Books Info ---
```

```
RangeIndex: 26 entries, 0 to 25
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	BookID	26 non-null	object
1	Title	26 non-null	object
2	Genre	26 non-null	object
3	Price	25 non-null	float64
4	AuthorID	26 non-null	object
5	PublishedYear	26 non-null	int64
6	CopiesSold	26 non-null	int64

```
dtypes: float64(1), int64(2), object(4)
```

```
memory usage: 1.5+ KB
```

```
--- Authors Info ---
```

```
RangeIndex: 6 entries, 0 to 5
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	AuthorID	6 non-null	object
1	AuthorName	6 non-null	object
2	Country	6 non-null	object
3	TotalBooks	6 non-null	int64

```
dtypes: int64(1), object(3)
```

```
memory usage: 320.0+ bytes
```

4. Handle Missing Values

```
print("Missing values count before handling:")
print(books_df.isnull().sum())

# Impute missing Price with median value
median_price = books_df['Price'].median()
books_df['Price'].fillna(median_price, inplace=True)

print("\nMissing values count after imputation:")
print(books_df.isnull().sum())
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #14

```
Missing values count before handling:
BookID          0
Title           0
Genre           0
Price           1
AuthorID        0
PublishedYear   0
CopiesSold      0

Missing values count after imputation:
BookID          0
Title           0
Genre           0
Price           0
AuthorID        0
PublishedYear   0
CopiesSold      0
```

5. Remove Duplicate Records

```
print(f"Duplicate rows detected: {books_df.duplicated().sum()}")
books_df.drop_duplicates(inplace=True)
print(f"Shape after removing duplicates: {books_df.shape}")
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #15

```
Duplicate rows detected: 1
Shape after removing duplicates: (25, 7)
```

6. Validate and Correct Data Types

```
# Correct data types: PublishedYear and CopiesSold to int
books_df['PublishedYear'] = books_df['PublishedYear'].astype(int)
books_df['CopiesSold'] = books_df['CopiesSold'].astype(int)
print("Corrected Data Types for Books:")
print(books_df.dtypes)
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #16

```
Corrected Data Types for Books:
BookID          object
Title           object
Genre           object
Price           float64
AuthorID        object
PublishedYear    int64
CopiesSold       int64
```

7. Merge the Two CSV Datasets Using the Common Key

```
# Merge using common key 'AuthorID'
merged_books = pd.merge(books_df, authors_df, on='AuthorID', how='inner')
print("Merged Dataset Preview (First 5 records):")
print(merged_books.head(5))
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #17

```
Merged Dataset Preview (First 5 records):
BookID      Title      Genre  Price AuthorID  PublishedYear  CopiesSold  AuthorName
Country  TotalBooks
B500 Book Title A  Fiction  37.79    A03         2011         1469 Stephen King
USA        64
B509 Book Title J  Sci-Fi  45.33    A03         2023         2059 Stephen King
USA        64
B514 Book Title O  Fiction  20.89    A03         2021         4161 Stephen King
USA        64
B522 Book Title W  Biography 37.70    A03         2002         4648 Stephen King
USA        64
B501 Book Title B  Mystery  45.22    A01         2005         2086 J.K. Rowling
UK         22
```

8. Perform Aggregation and Summary Statistics

```
print("--- Summary Statistics for Merged Dataset ---")
print(merged_books.describe())

print("\n--- Aggregation: Total & Average Copies Sold by Genre ---")
genre_agg = merged_books.groupby('Genre')['CopiesSold'].agg(['mean', 'sum', 'count'])
print(genre_agg)
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #18

```
--- Summary Statistics for Merged Dataset ---
      Price  PublishedYear  CopiesSold  TotalBooks
count  25.00000      25.000000    25.000000    25.000000
mean   26.96760     2011.280000   2538.600000    82.760000
std    10.06941       7.807689   1337.929931   122.395289
min    10.02000     2000.000000    246.000000    15.000000
25%    20.78000     2005.000000   1834.000000    22.000000
50%    24.10000     2010.000000   2485.000000    22.000000
75%    34.97000     2019.000000   3469.000000    85.000000
max    46.27000     2023.000000   4836.000000   400.000000

--- Aggregation: Total & Average Copies Sold by Genre ---
              mean      sum  count
Genre
Biography   2964.750000  11859     4
Fiction     2929.250000  11717     4
History     2396.000000  19168     8
Mystery     2372.333333  14234     6
Sci-Fi      2162.333333   6487     3
```

9. Identify the Category with the Highest Value

```
highest_sold_genre = genre_agg['sum'].idxmax()
highest_sold_value = genre_agg['sum'].max()
print(f"Genre Category with the Highest Total Copies Sold: {highest_sold_genre}
({highest_sold_value:,} copies)")
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #19

```
Genre Category with the Highest Total Copies Sold: History (19,168 copies)
```

10. Export the Cleaned Dataset to a New CSV File

```
output_filename = 'Cleaned_Library_Records.csv'  
merged_books.to_csv(output_filename, index=False)  
print(f"Confirmation: Cleaned dataset successfully exported to '{output_filename}')
```

EXECUTION CONSOLE OUTPUT - SCREENSHOT #20

```
Confirmation: Cleaned dataset successfully exported to 'Cleaned_Library_Records.csv'
```